

Sorting

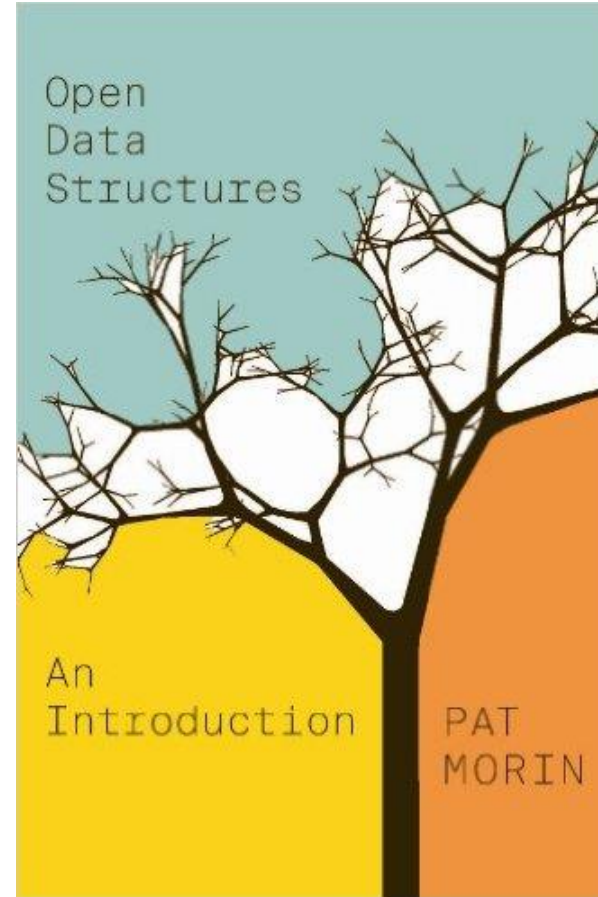
COMP 2402

Abstract Data Types & Algorithms

Readings

Today's topic

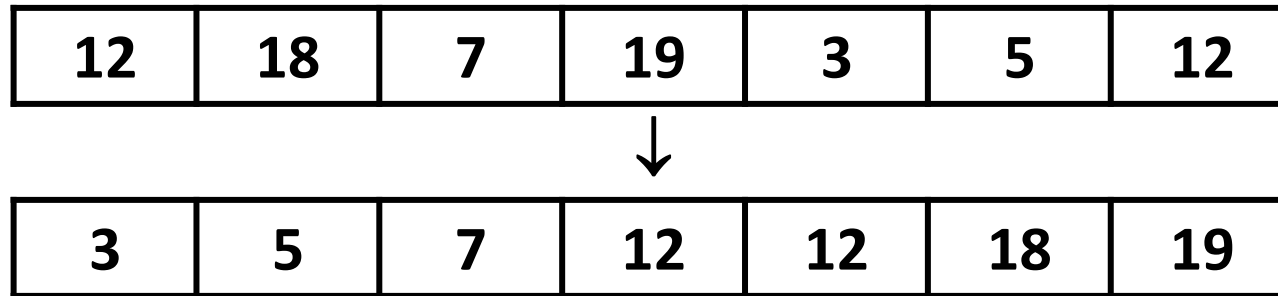
- Comparison-Based Sorting
- Heapsort



Sorting

Comparison-Based Sorting

We'll look at several algorithms that order an array of items in non-decreasing order.



The (re)arrangement of the items is accomplished by repeatedly **comparing two items** in the array.

Sorting

Comparison-Based Sorting

The data we are sorting must come from a **total order**.

For any two elements x, y

$$\begin{array}{l} x.\text{compareTo}(y) \\ \text{compare}(x, y) \end{array} = \begin{cases} < 0 & \text{when } x < y \\ 0 & \text{when } x = y \\ > 0 & \text{when } x > y \end{cases}$$

Sorting

With **Comparison-Based** sorting algorithms, we base the efficiency on the total number of **comparisons** needed to sort the data.

An **in-place** algorithm requires only $O(1)$ extra space. In-place sorting algorithms use the array and swap elements.

A **stable** sorting algorithms maintain the relative ordering of repeated values.

Sorting

Basic quadratic sorting algorithms.

- Insertion sort
- Selection sort
- Bubble sort

All of these can be seen as specific instances of a more general algorithm...

Sorting

```
generalSort(U, S) :
```

```
    invariant: U is unsorted, S is sorted
```

```
    for each x in U do:
```

```
        remove x from U
```

```
        add x to S
```

```
    output S
```

Sorting

`generalSort(U, S) :`

invariant: U is unsorted, S is sorted

`for each x in U do:`

`remove x from U`

`add x to S`

`output S`

`generalSort(A, Empty)`
will sort the list A

Sorting

generalSort (U, S) :

invariant: U is unsorted, S is sorted

for each x in U do:

remove x from U

add x to S

output S

details of choosing the next x and how to add to S differentiate the specific algorithms

Sorting

InsertionSort

Sorting

SelectionSort

Sorting

BubbleSort

Sorting

	# comparison	in-place	stable
Insertion sort	$O(n) \rightarrow O(n^2)$	✓	✓
Selection sort	$O(n^2)$	✓	✓
Bubble sort	$O(n) \rightarrow O(n^2)$	✓	✓

All of these are quadratic in the worst case.

Both insertion sort and bubble sort are **adaptive** (they are faster when the data has some initial structure)

Sorting

Can we do better than worst-case quadratic runtime?

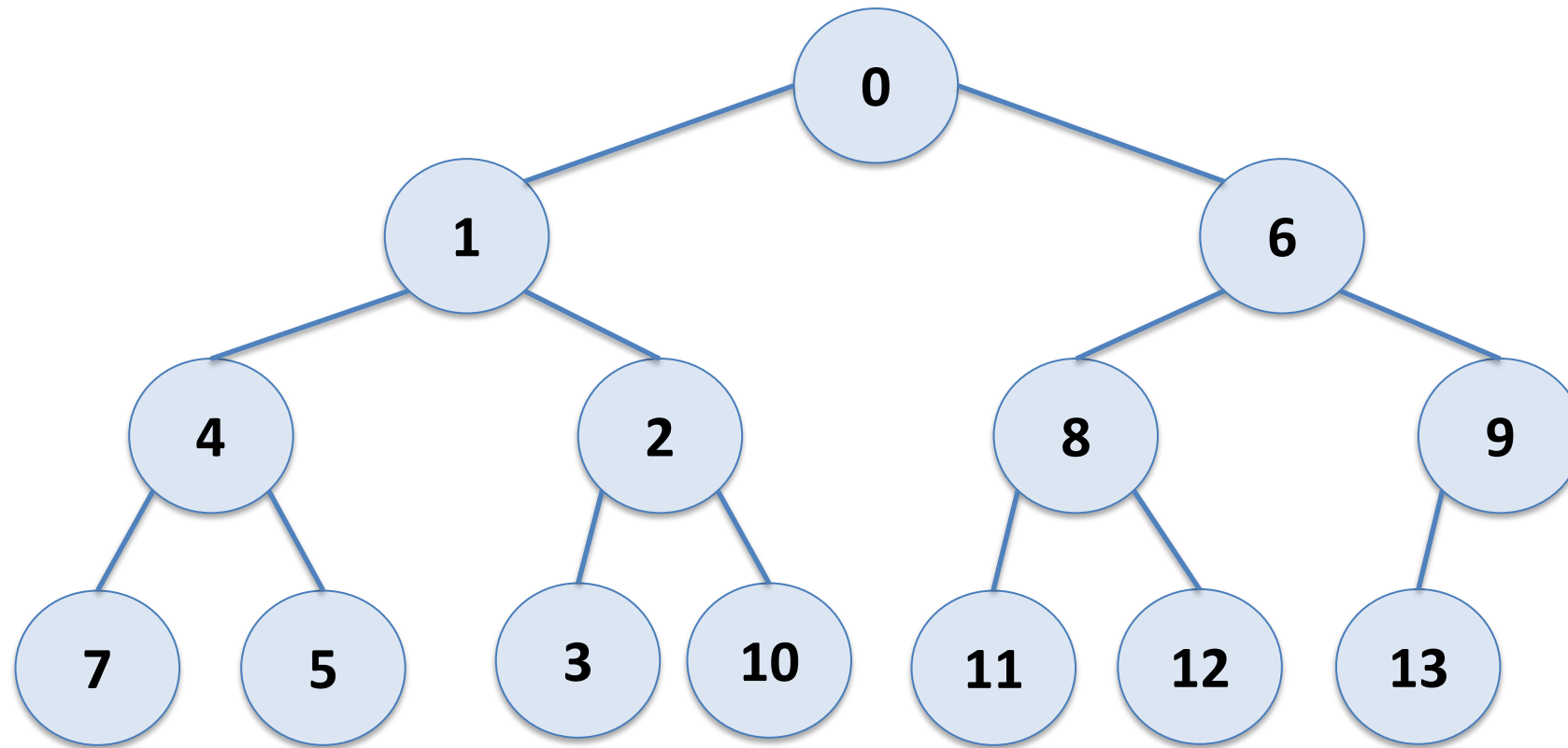
Hint: lots of programs would be really slow if not.

Heapsort

Question: Given a binary heap (where priority is just the data itself) can you sort the data in the heap?

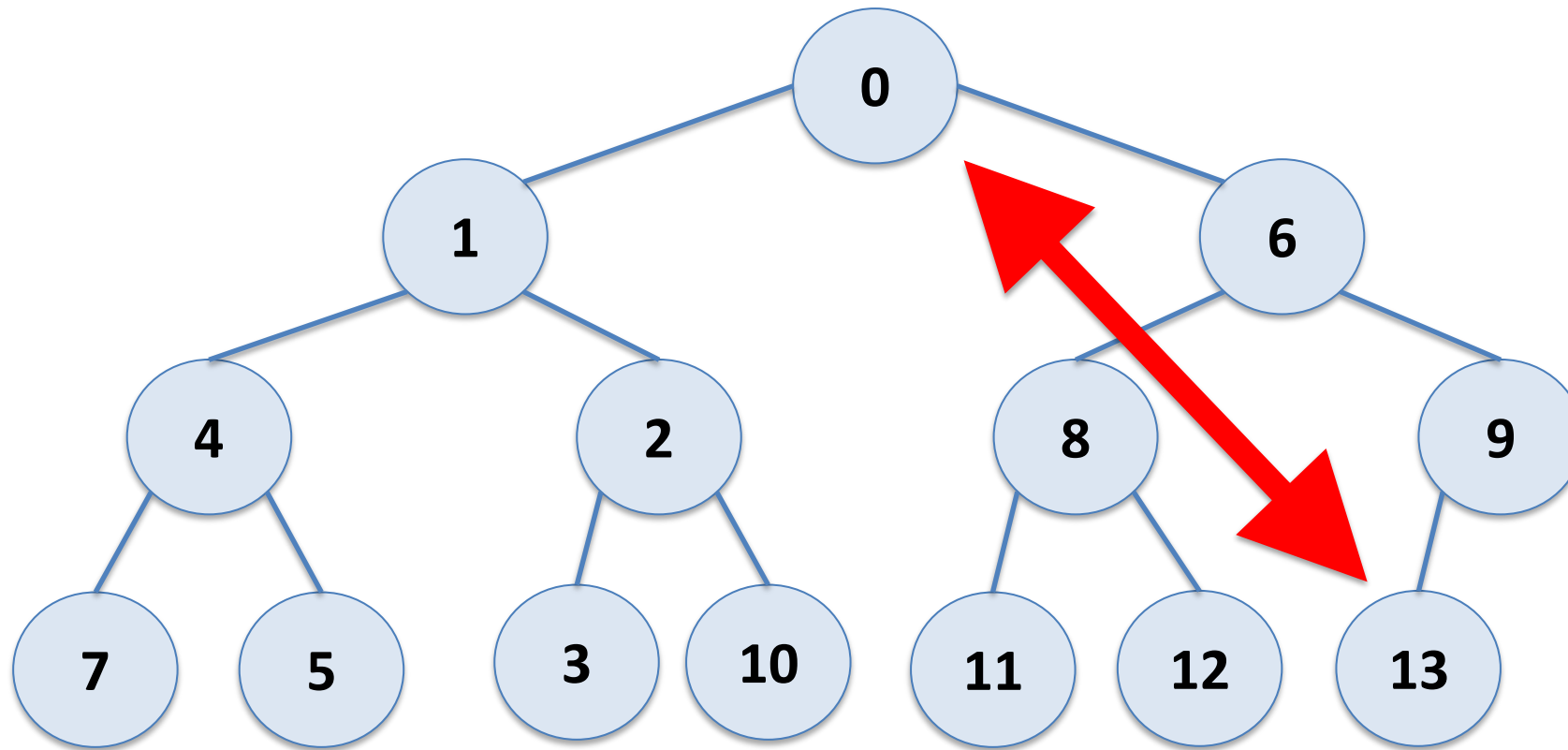
Sorting a Heap

0	1	6	4	2	8	9	7	5	3	10	11	12	13	
---	---	---	---	---	---	---	---	---	---	----	----	----	----	--



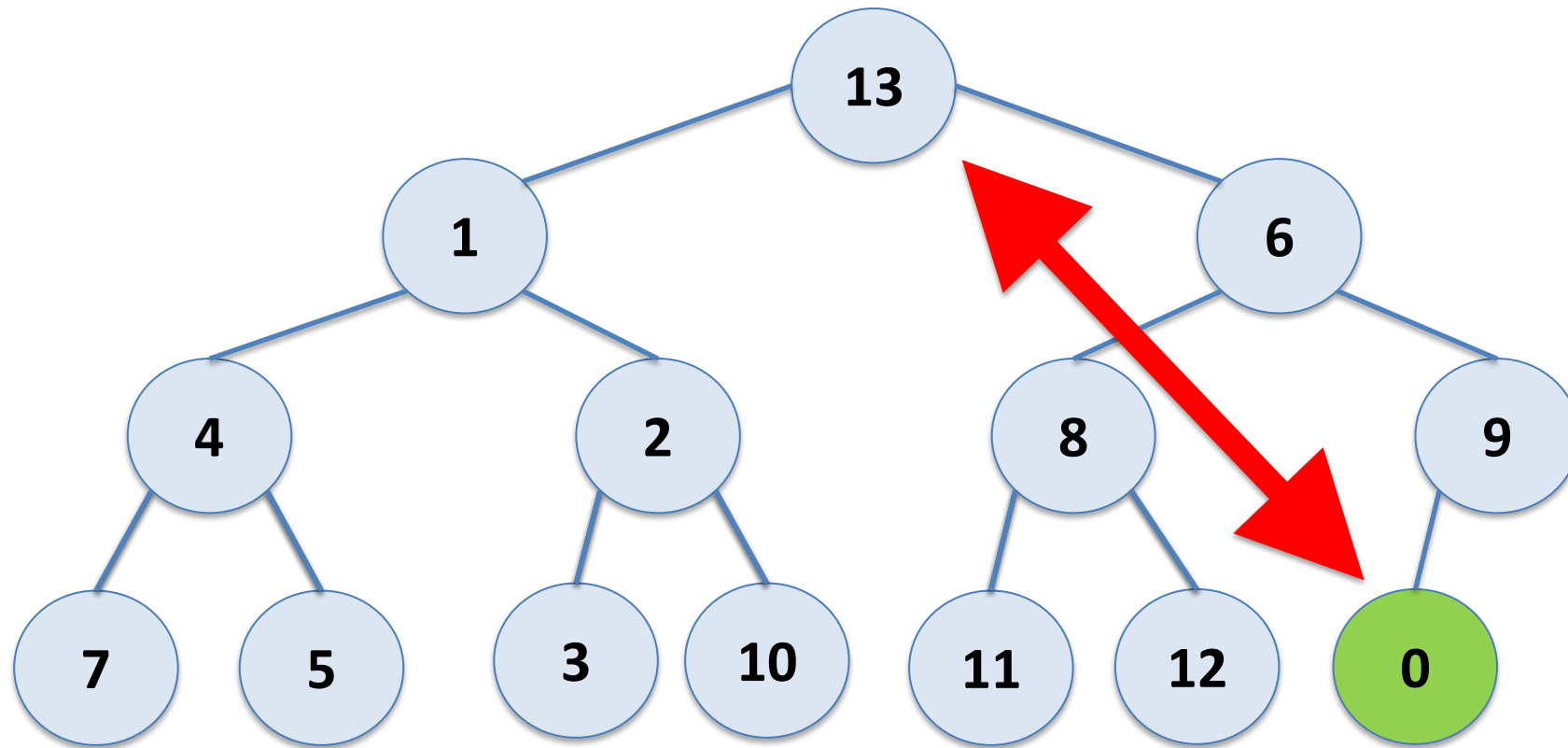
Sorting a Heap

0	1	6	4	2	8	9	7	5	3	10	11	12	13	
---	---	---	---	---	---	---	---	---	---	----	----	----	----	--



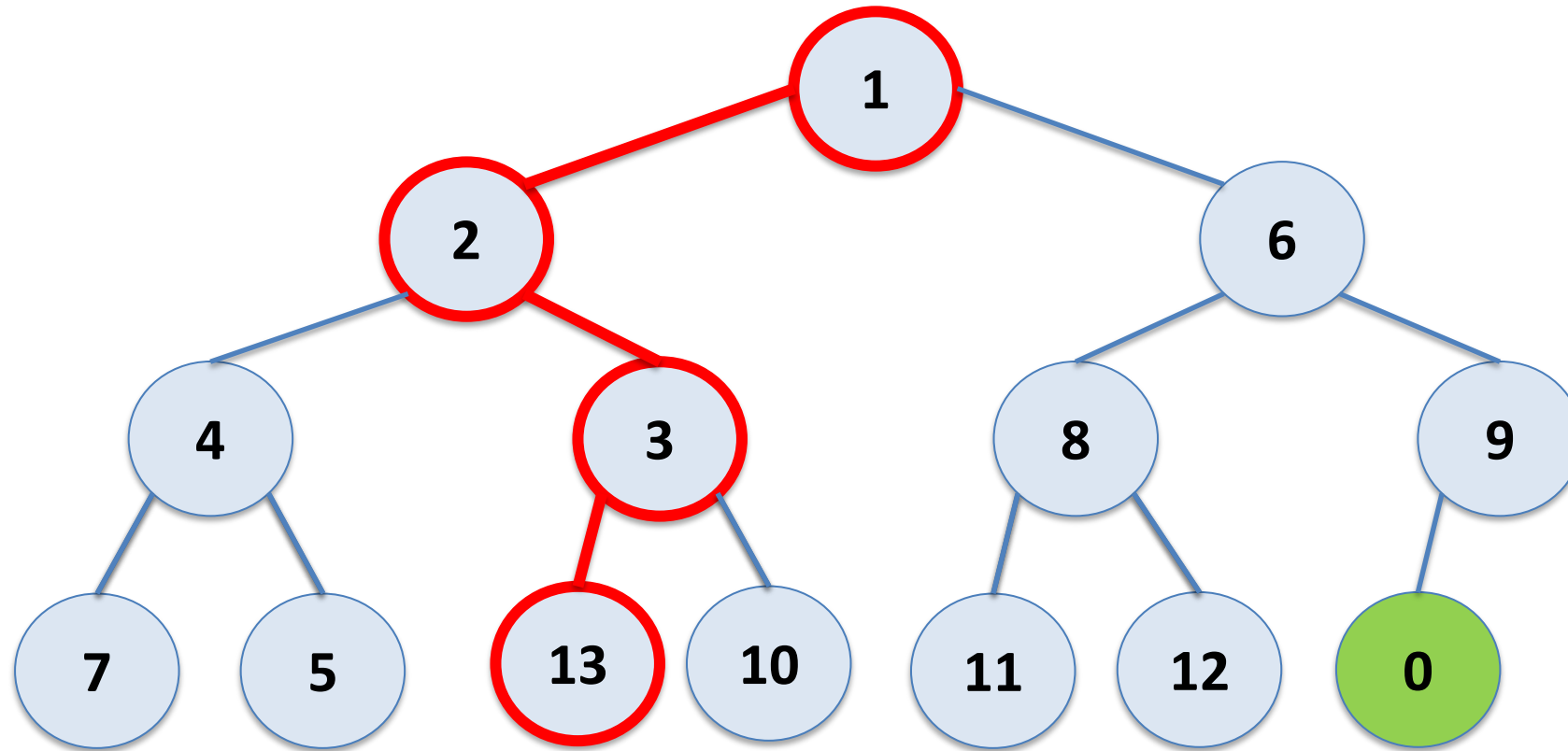
Sorting a Heap

13	1	6	4	2	8	9	7	5	3	10	11	12	0	
----	---	---	---	---	---	---	---	---	---	----	----	----	---	--



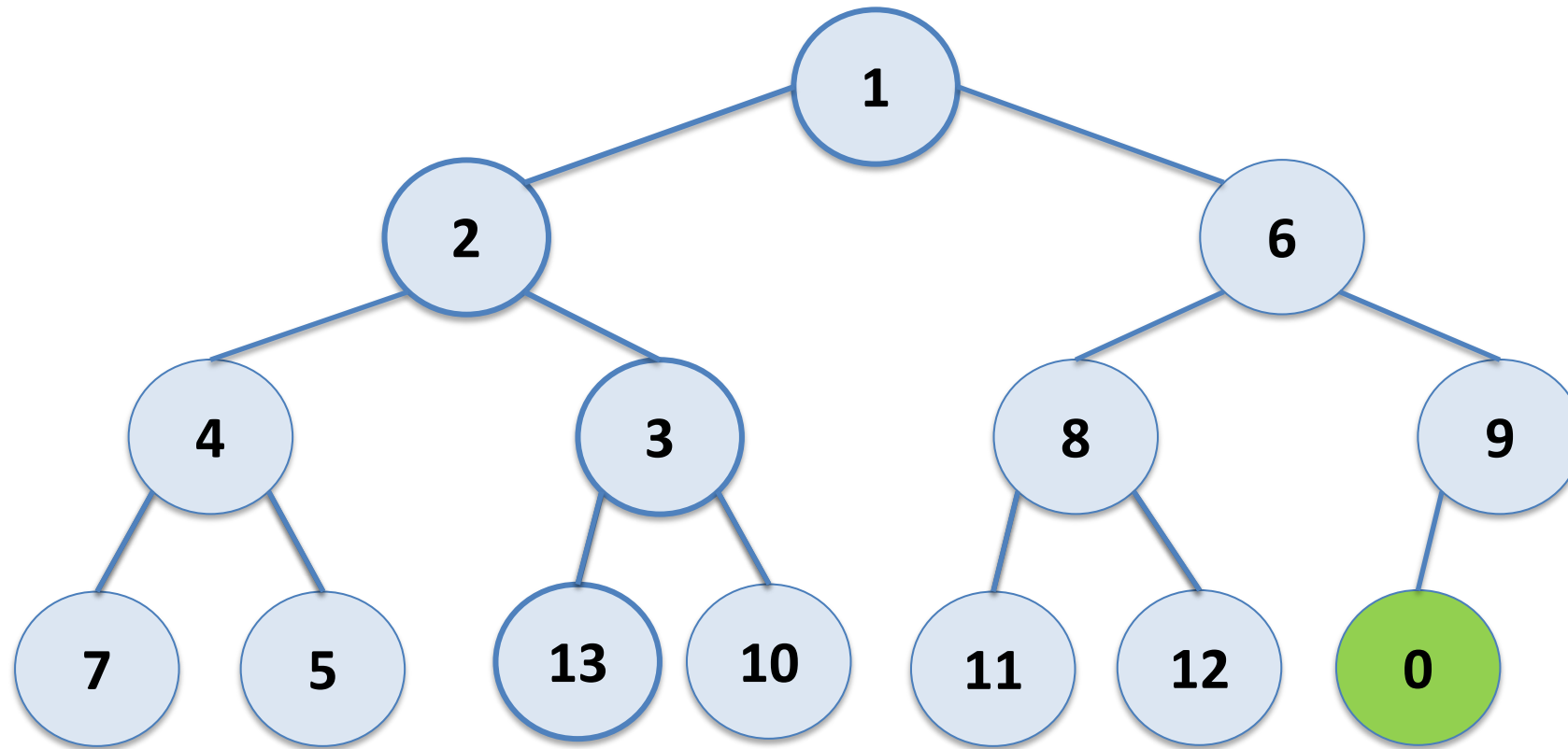
Sorting a Heap

1	2	6	4	3	8	9	7	5	13	10	11	12	0	
---	---	---	---	---	---	---	---	---	----	----	----	----	---	--



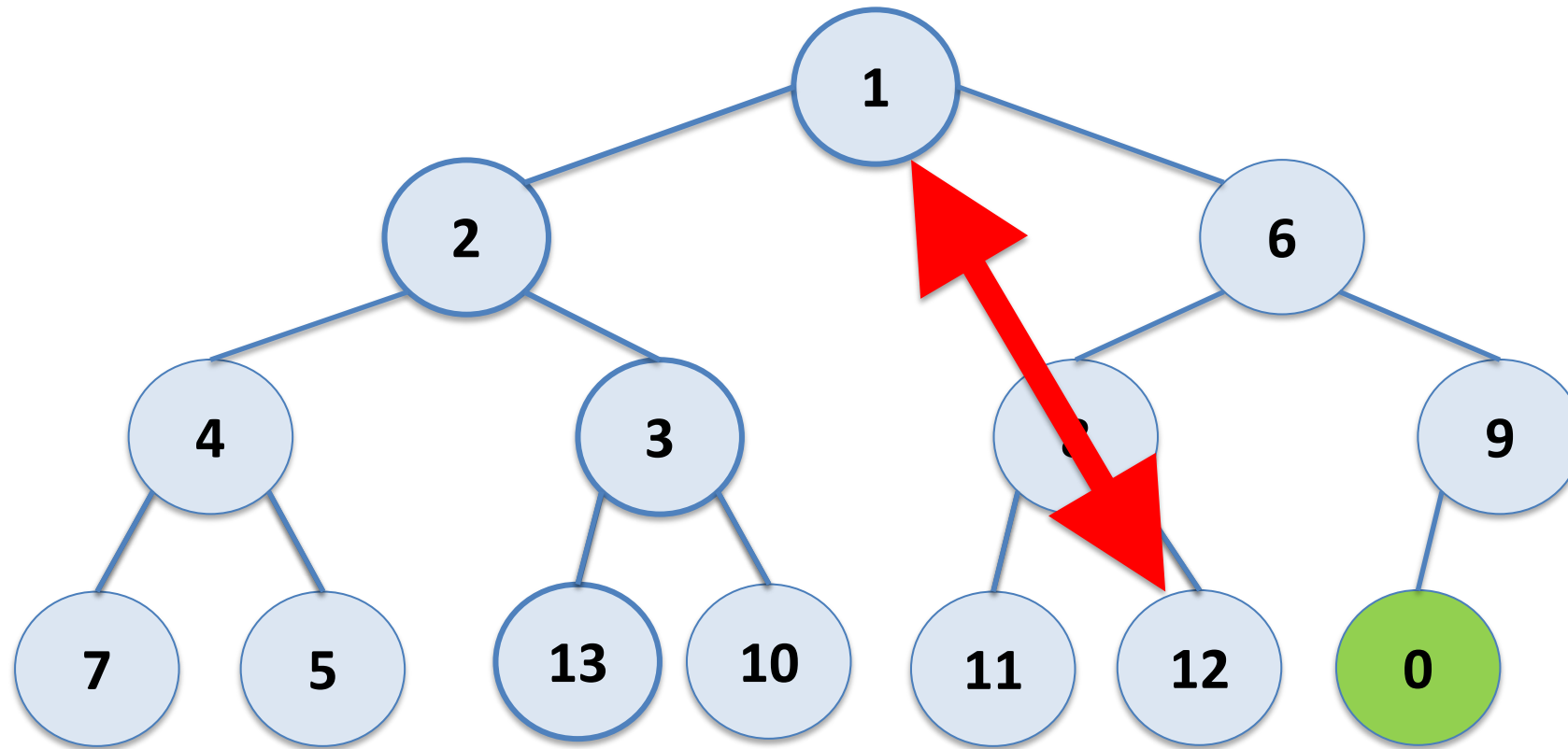
Sorting a Heap

1	2	6	4	3	8	9	7	5	13	10	11	12	0	
---	---	---	---	---	---	---	---	---	----	----	----	----	---	--



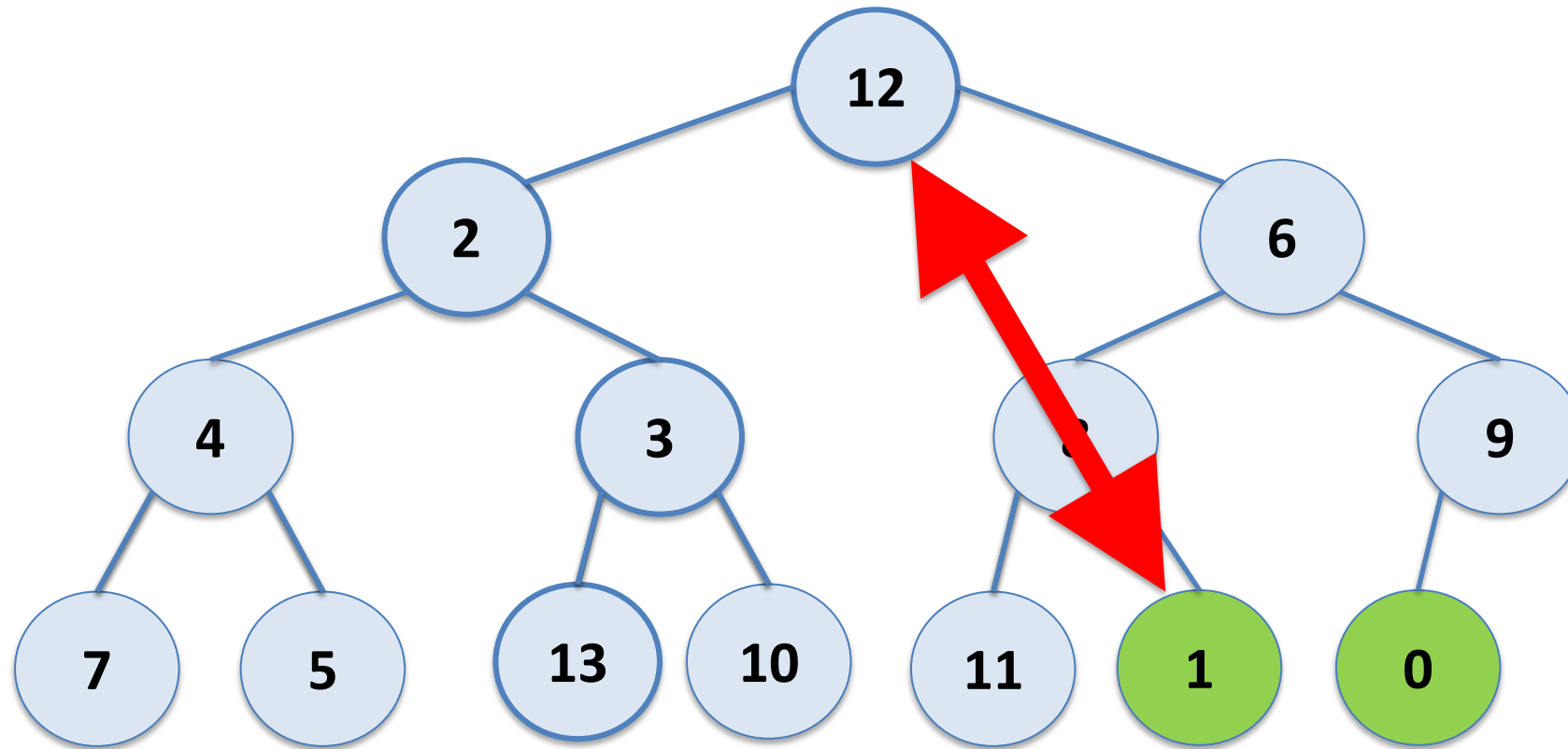
Sorting a Heap

1	2	6	4	3	8	9	7	5	13	10	11	12	0	
---	---	---	---	---	---	---	---	---	----	----	----	----	---	--



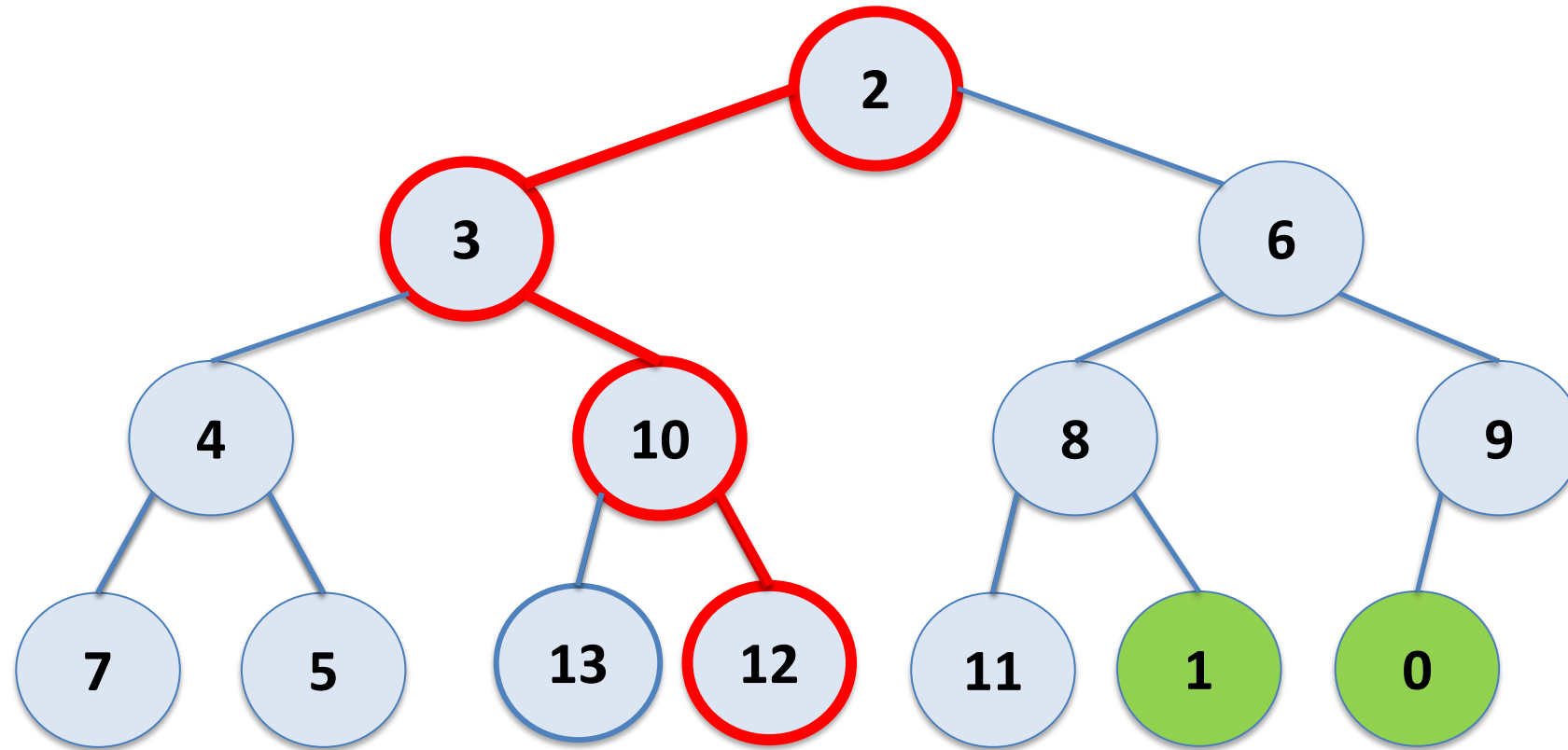
Sorting a Heap

12	2	6	4	3	8	9	7	5	13	10	11	1	0	
----	---	---	---	---	---	---	---	---	----	----	----	---	---	--



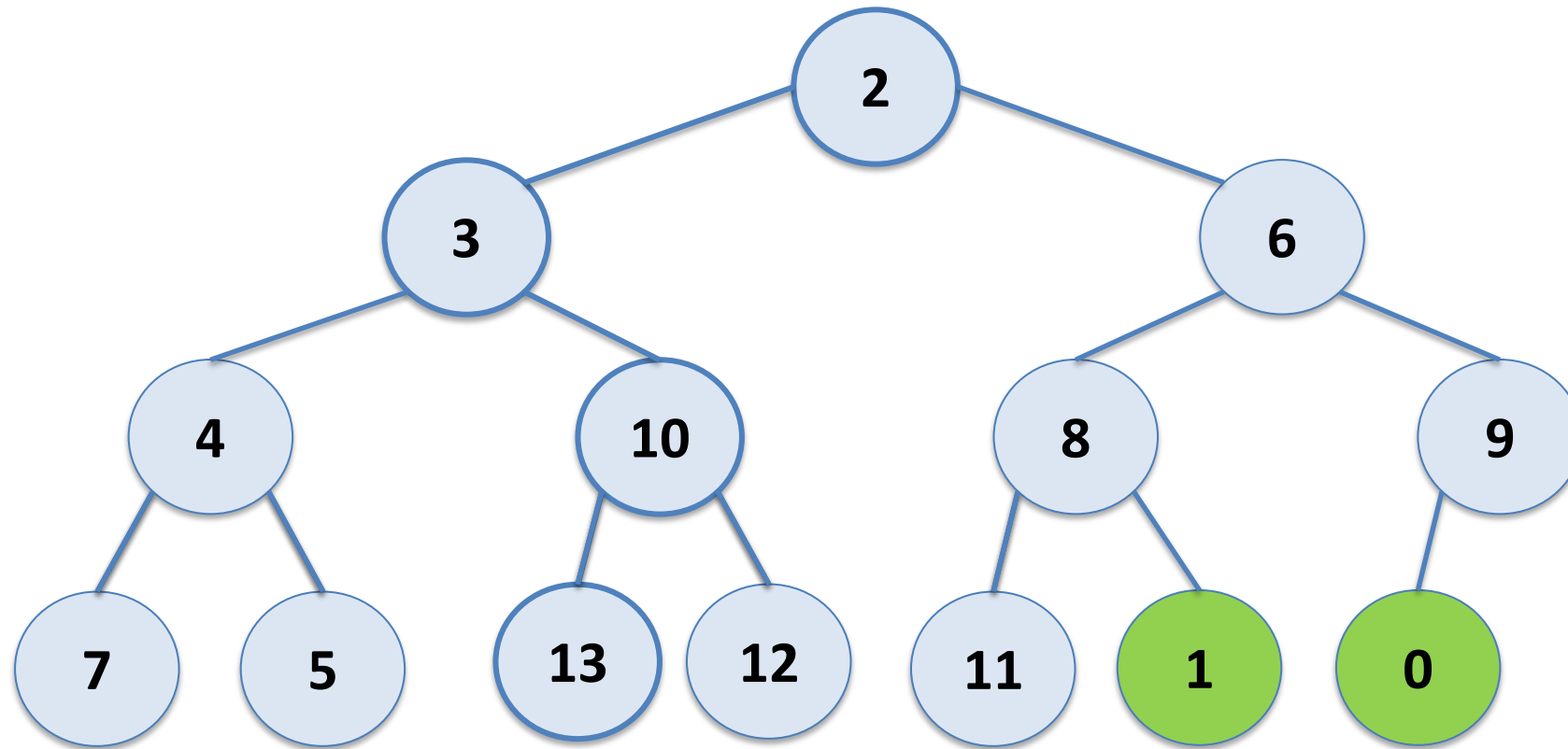
Sorting a Heap

2	3	6	4	10	8	9	7	5	13	12	11	1	0	
---	---	---	---	----	---	---	---	---	----	----	----	---	---	--



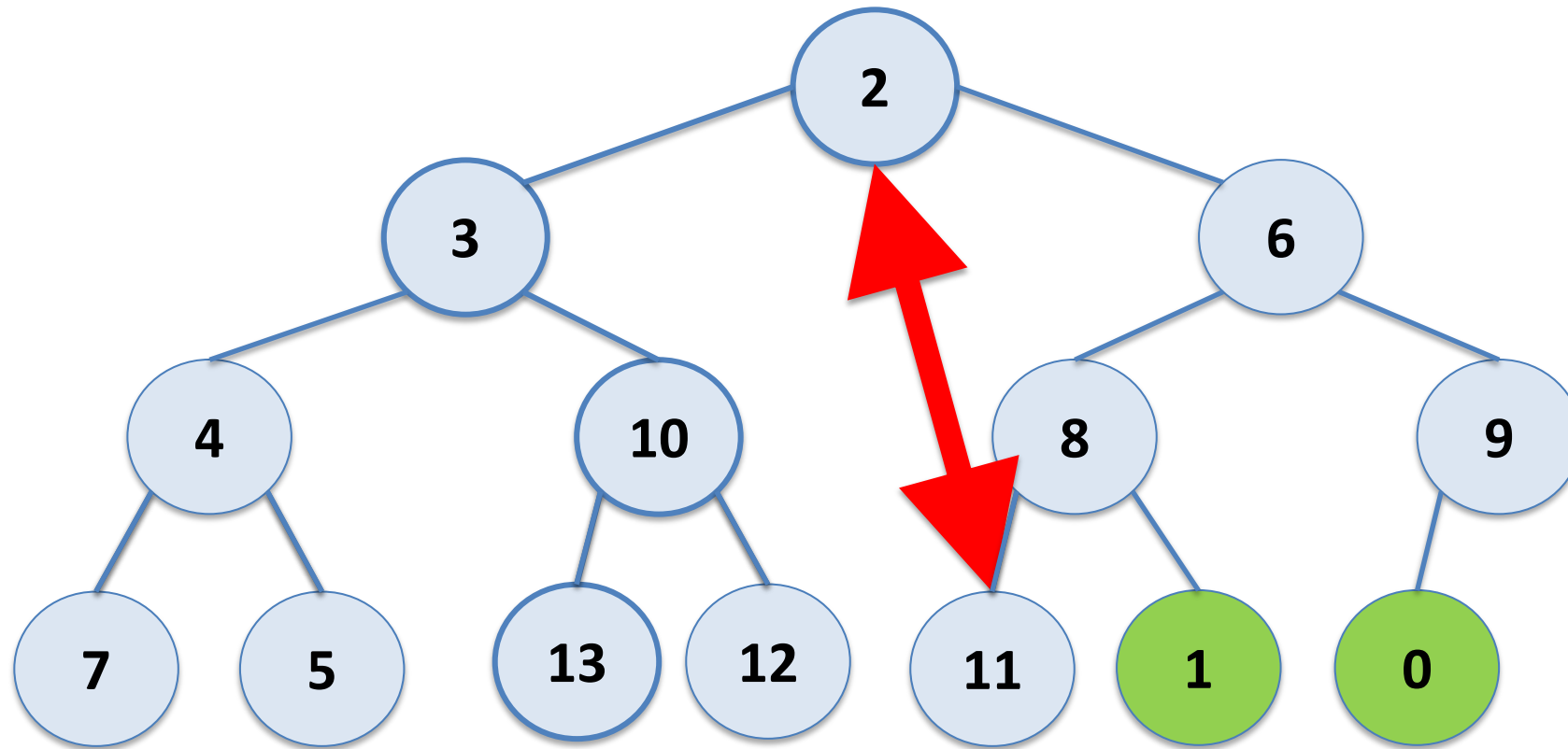
Sorting a Heap

2	3	6	4	10	8	9	7	5	13	12	11	1	0	
---	---	---	---	----	---	---	---	---	----	----	----	---	---	--



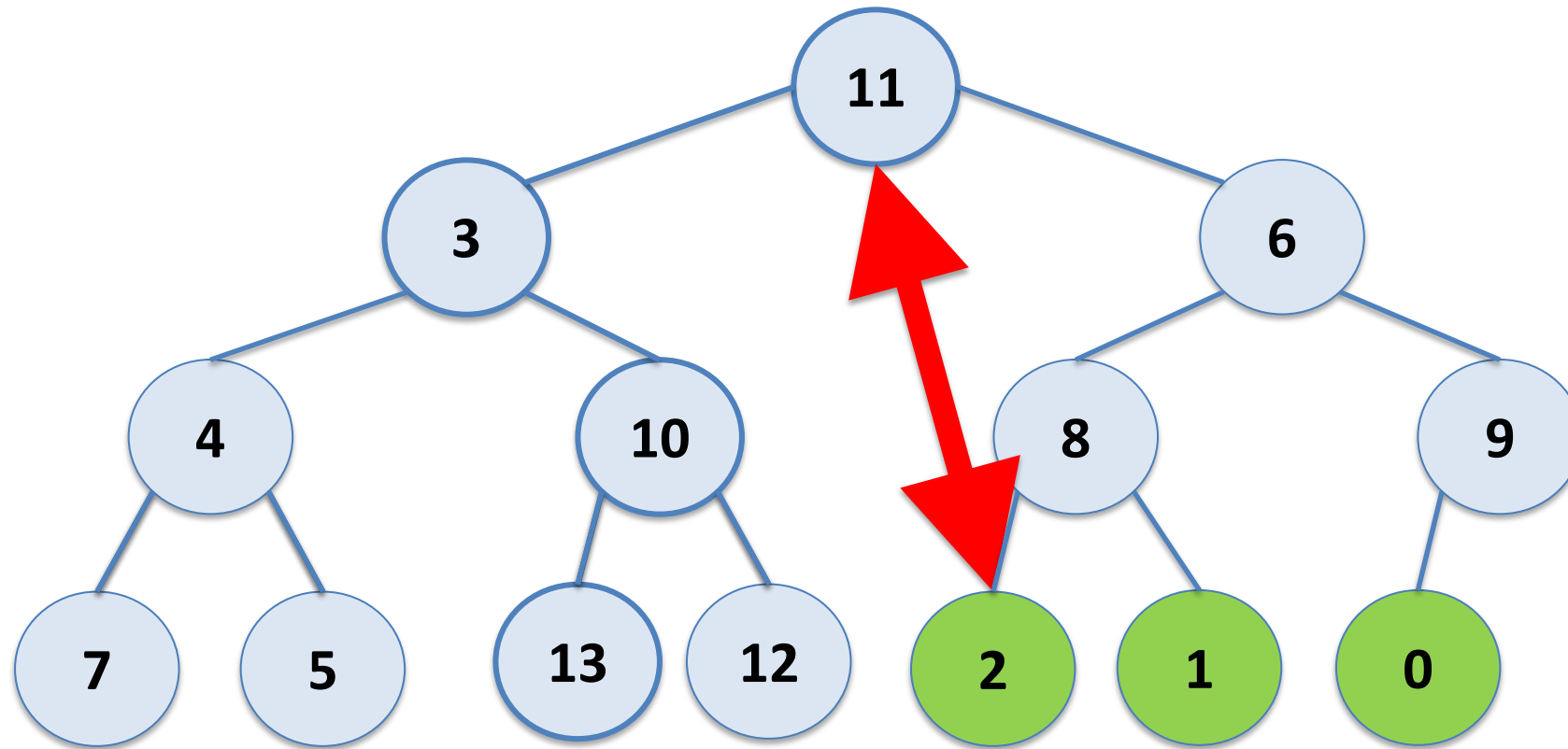
Sorting a Heap

2	3	6	4	10	8	9	7	5	13	12	11	1	0	
---	---	---	---	----	---	---	---	---	----	----	----	---	---	--



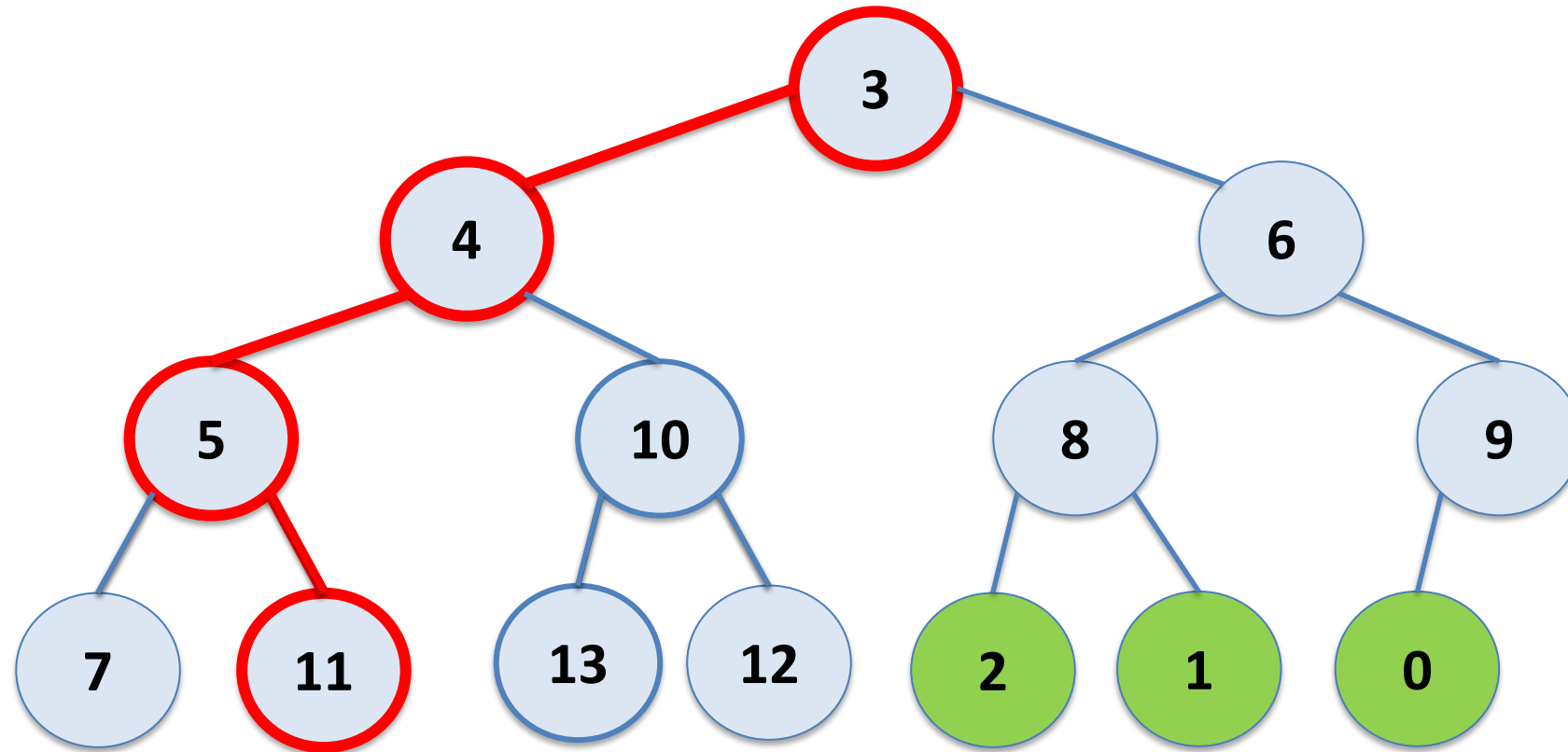
Sorting a Heap

11	3	6	4	10	8	9	7	5	13	12	2	1	0	
----	---	---	---	----	---	---	---	---	----	----	---	---	---	--



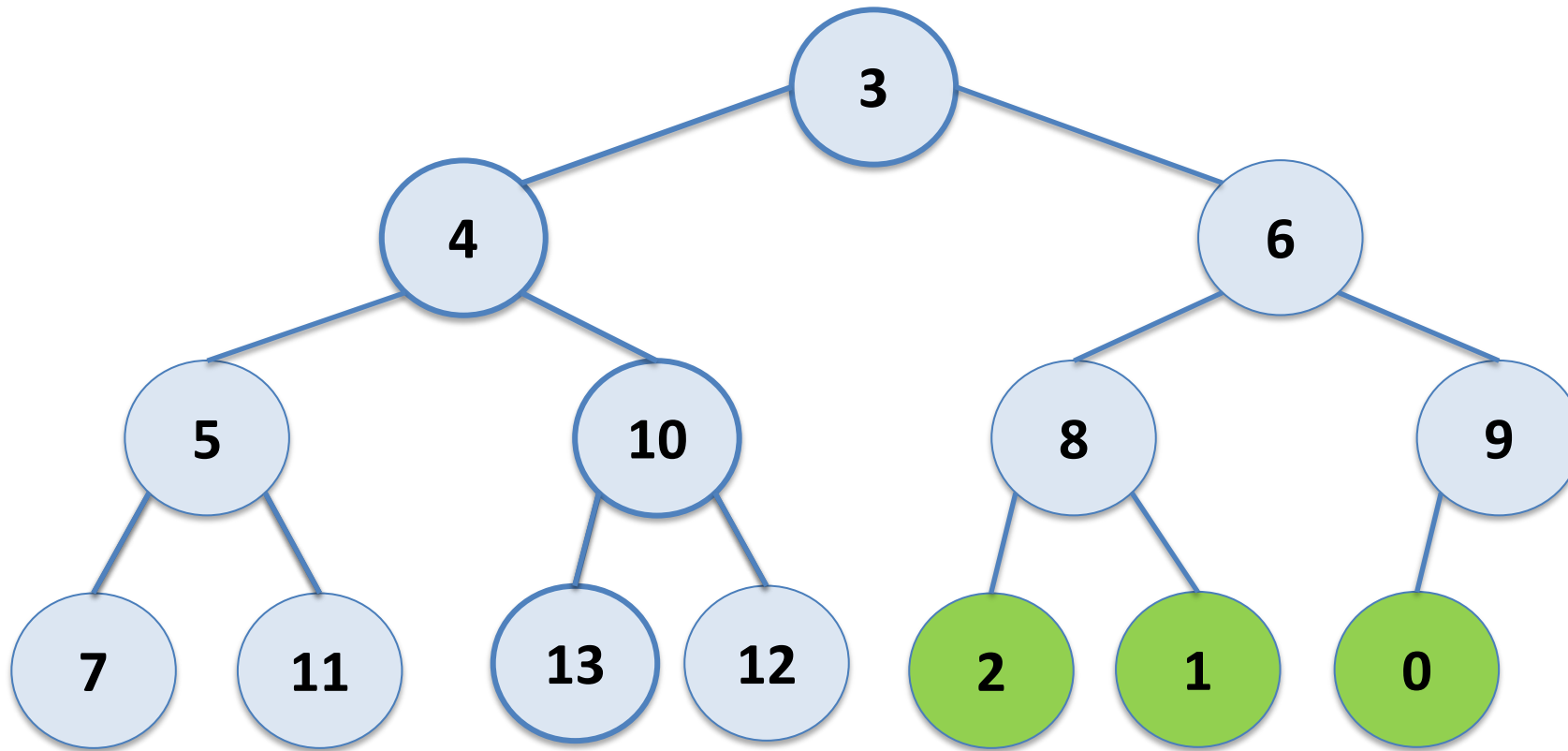
Sorting a Heap

3	4	6	5	10	8	9	7	11	13	12	2	1	0	
---	---	---	---	----	---	---	---	----	----	----	---	---	---	--



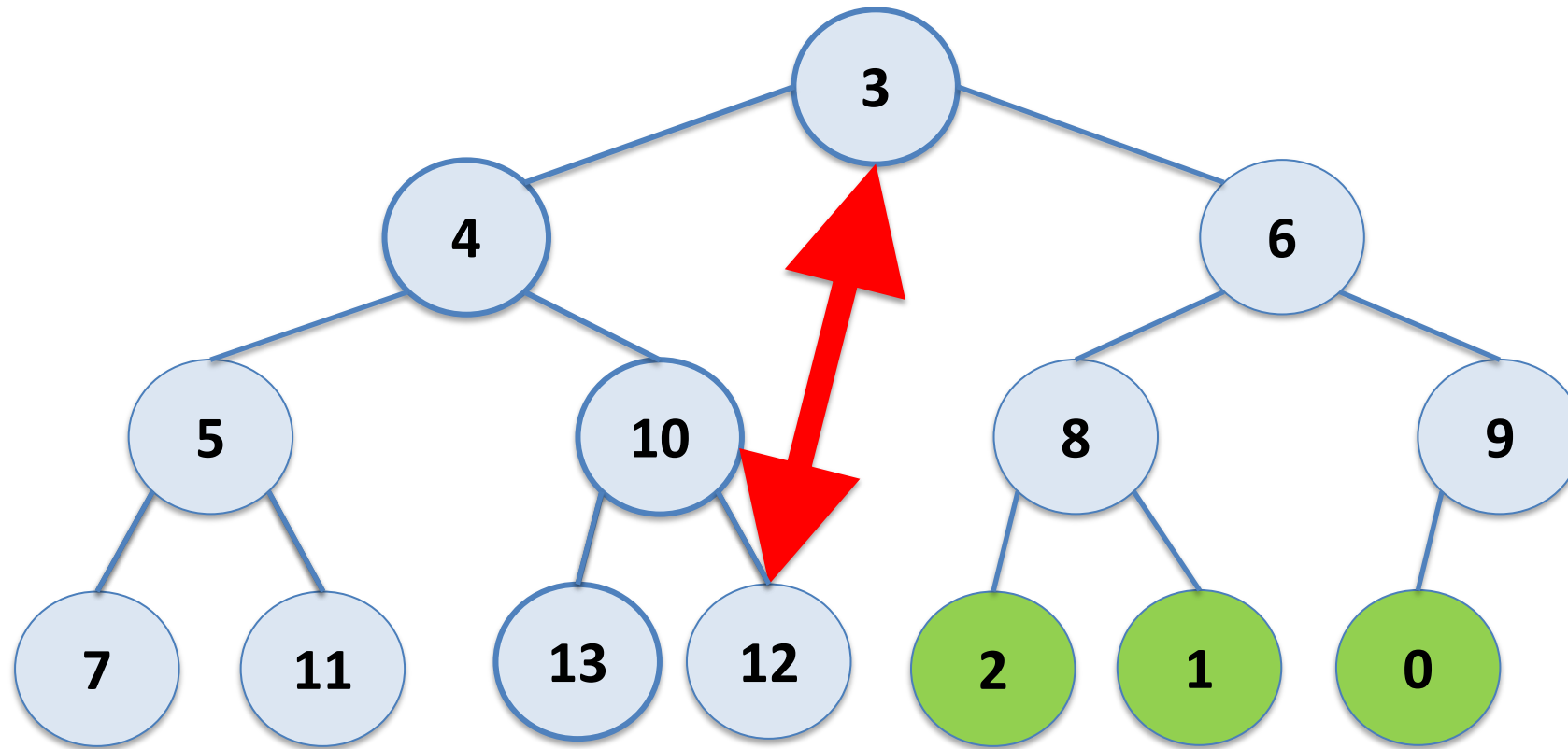
Sorting a Heap

3	4	6	5	10	8	9	7	11	13	12	2	1	0	
---	---	---	---	----	---	---	---	----	----	----	---	---	---	--



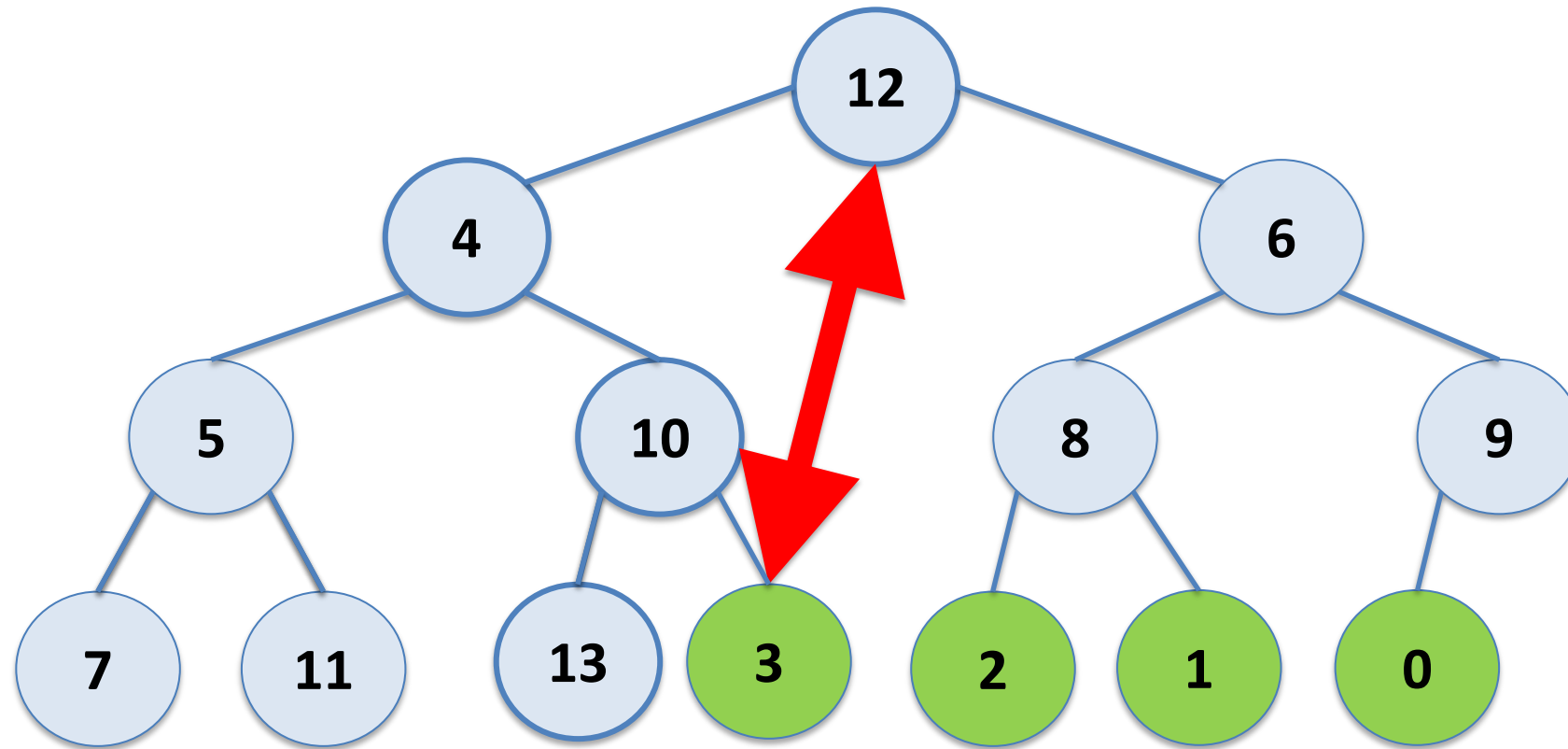
Sorting a Heap

3	4	6	5	10	8	9	7	11	13	12	2	1	0	
---	---	---	---	----	---	---	---	----	----	----	---	---	---	--



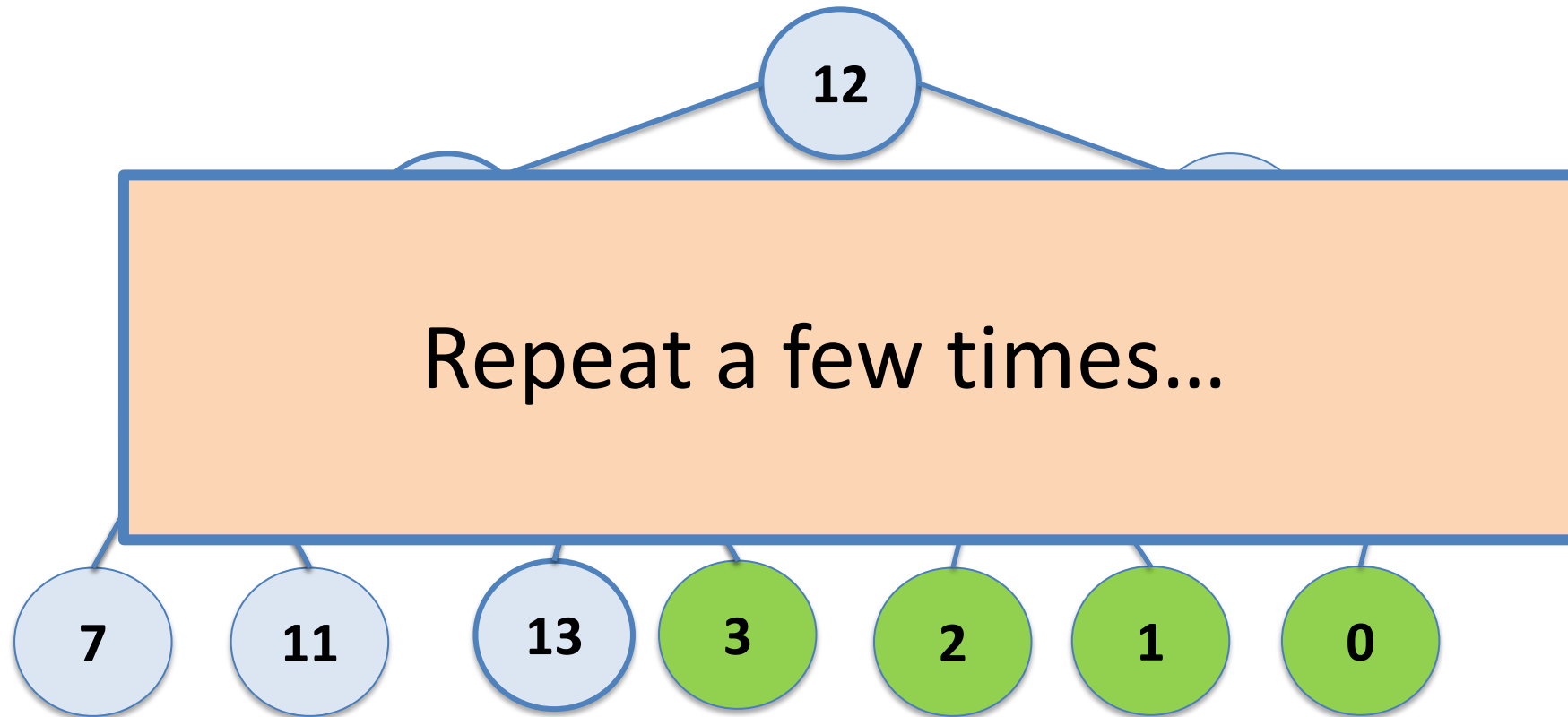
Sorting a Heap

12	4	6	5	10	8	9	7	11	13	3	2	1	0	
----	---	---	---	----	---	---	---	----	----	---	---	---	---	--



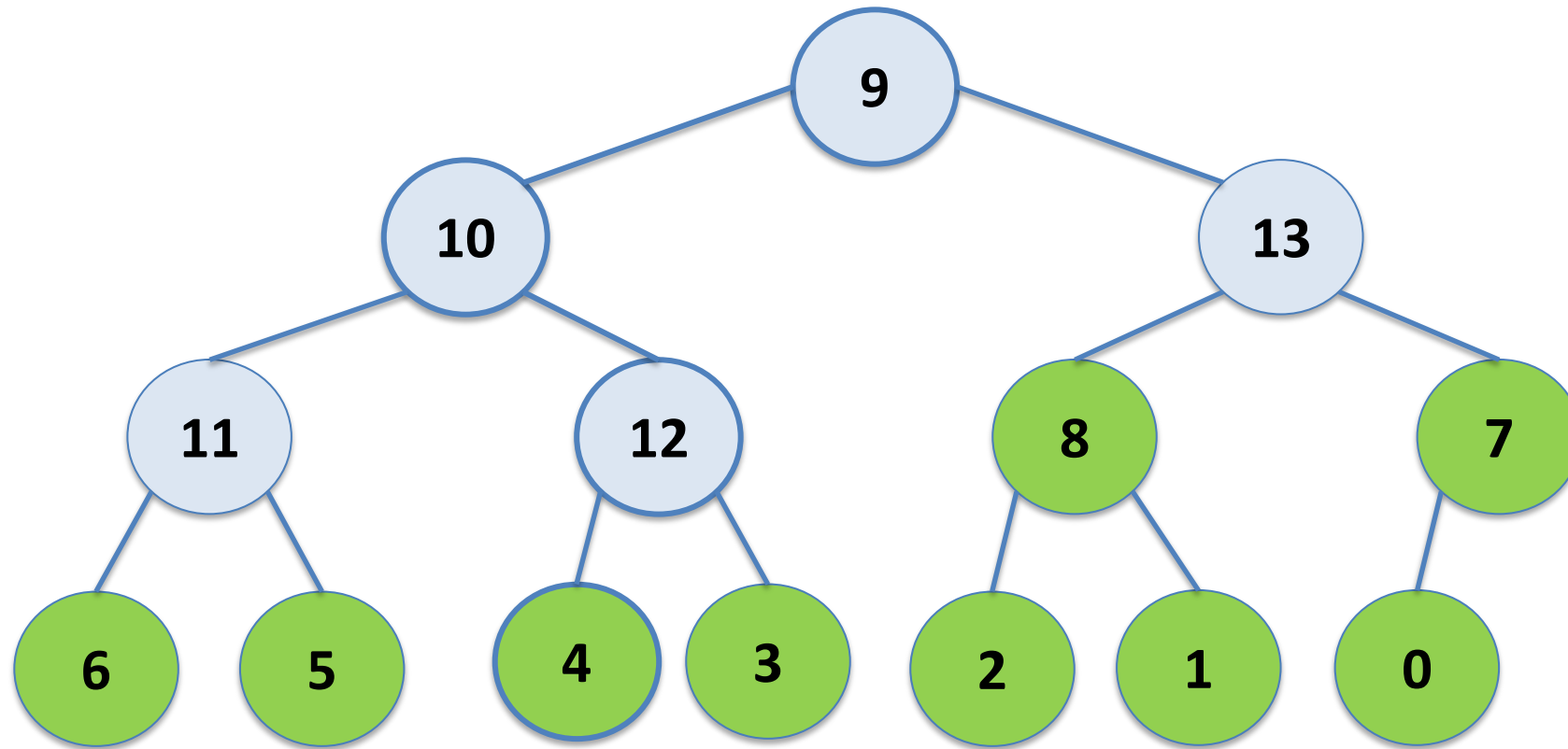
Sorting a Heap

12	4	6	5	10	8	9	7	11	13	3	2	1	0	
----	---	---	---	----	---	---	---	----	----	---	---	---	---	--



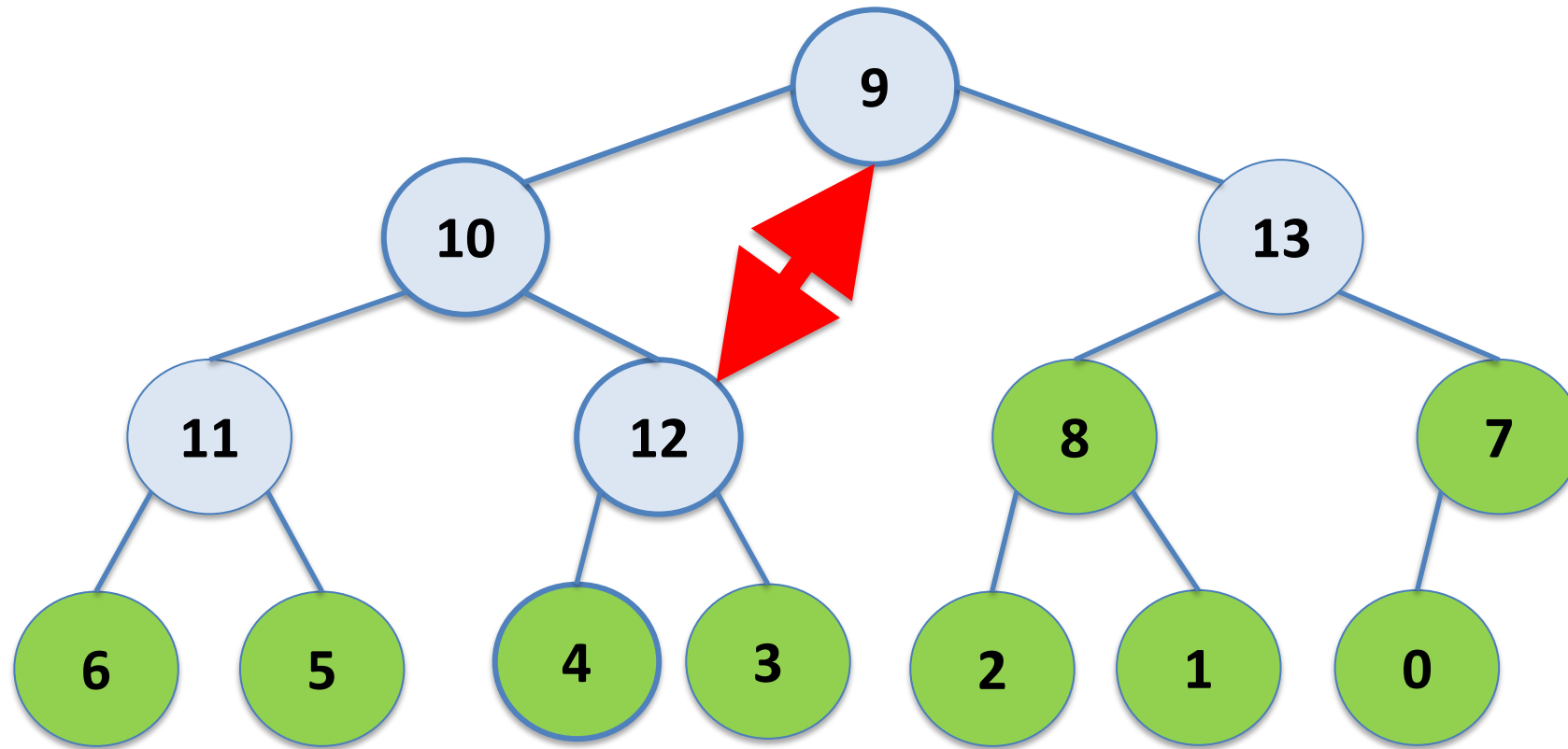
Sorting a Heap

9	10	13	11	12	8	7	6	5	4	3	2	1	0	
---	----	----	----	----	---	---	---	---	---	---	---	---	---	--



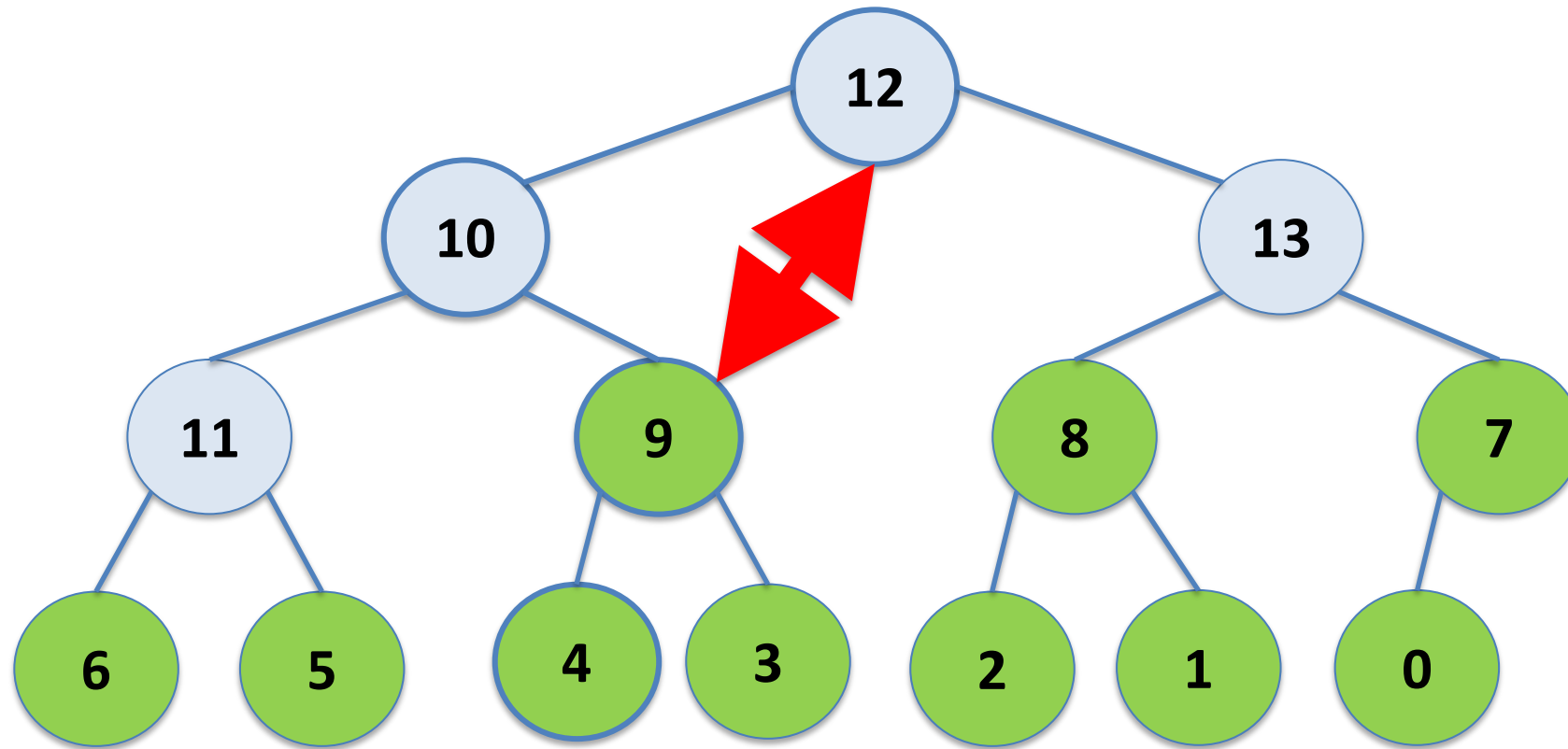
Sorting a Heap

9	10	13	11	12	8	7	6	5	4	3	2	1	0	
---	----	----	----	----	---	---	---	---	---	---	---	---	---	--



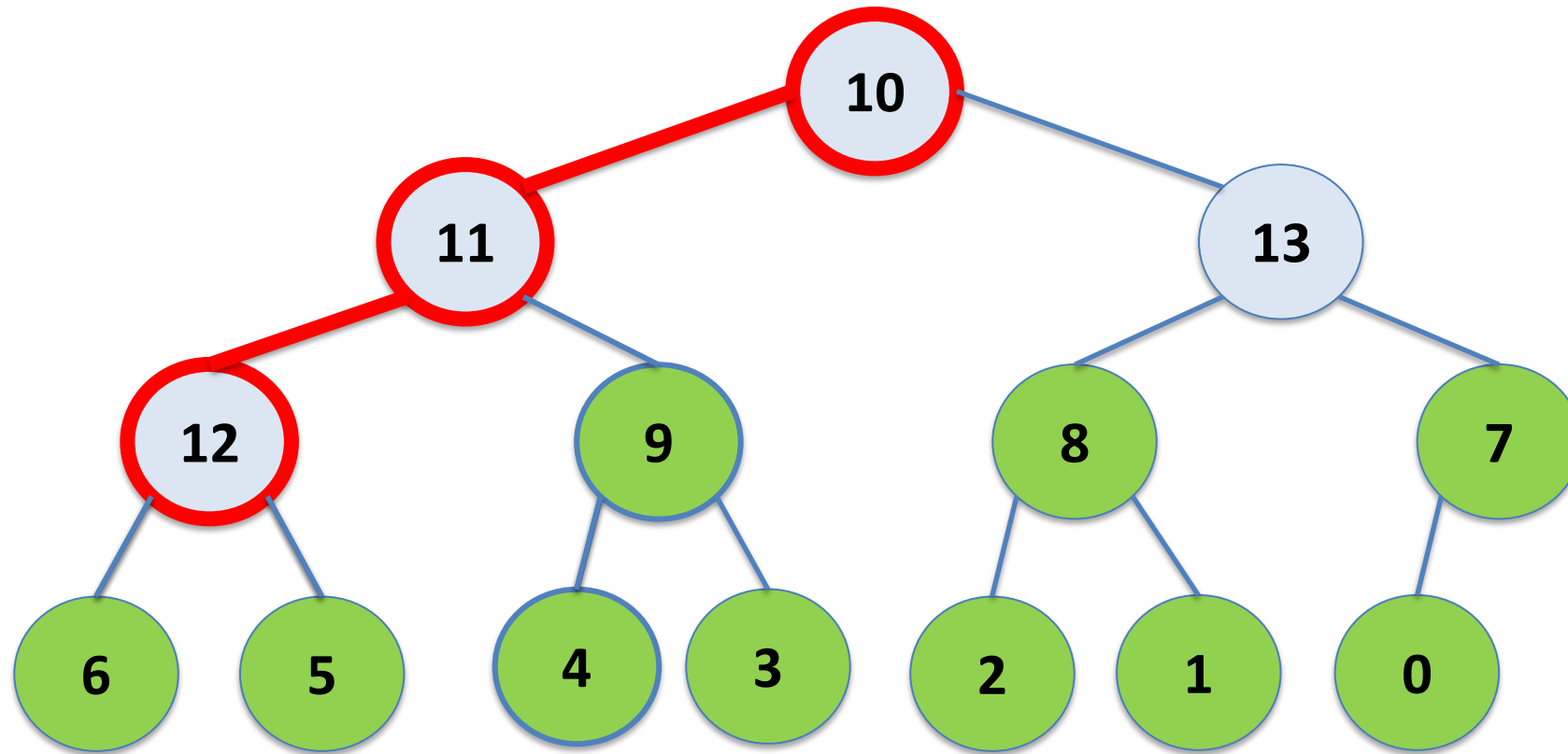
Sorting a Heap

12	10	13	11	9	8	7	6	5	4	3	2	1	0	
----	----	----	----	---	---	---	---	---	---	---	---	---	---	--



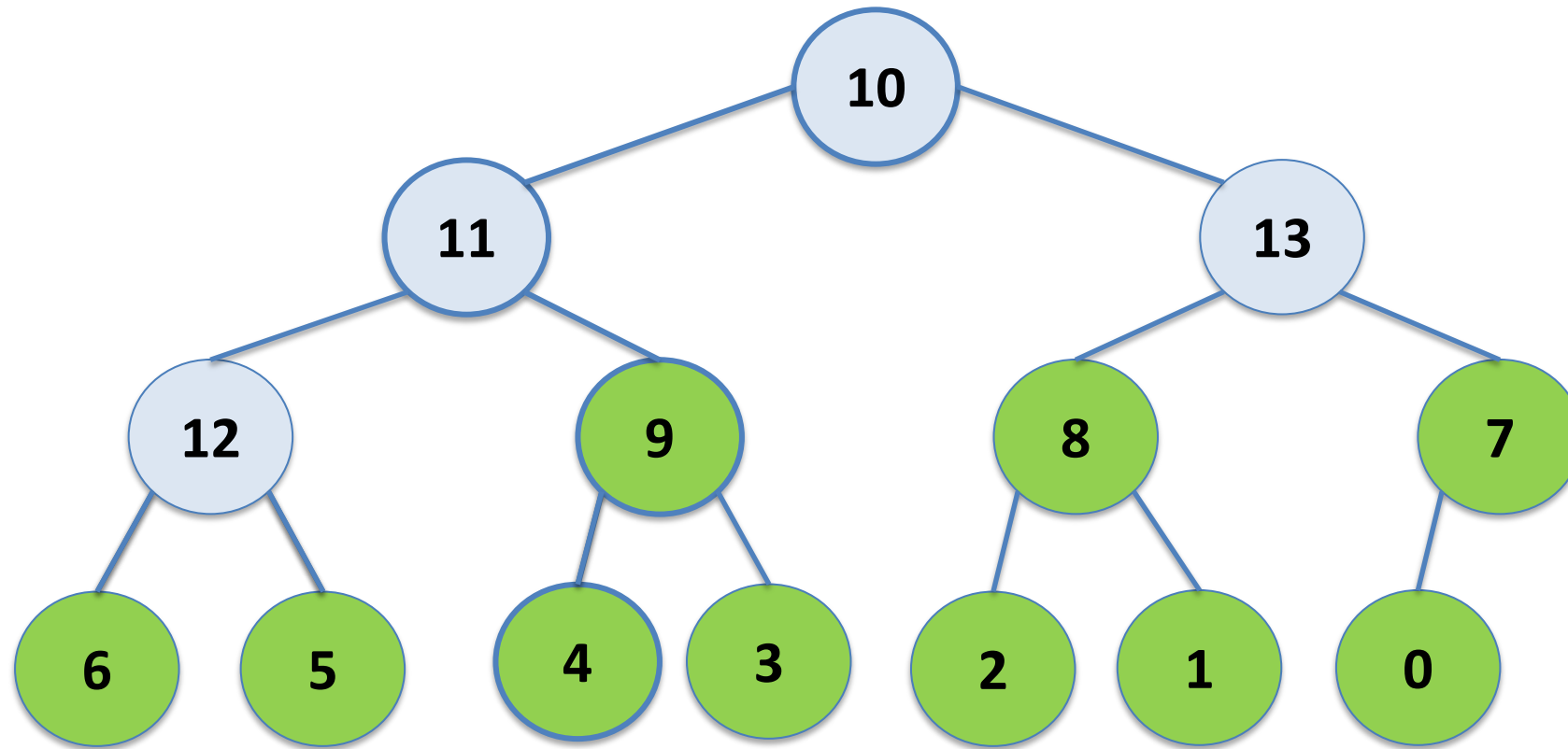
Sorting a Heap

10	11	13	12	9	8	7	6	5	4	3	2	1	0	
----	----	----	----	---	---	---	---	---	---	---	---	---	---	--



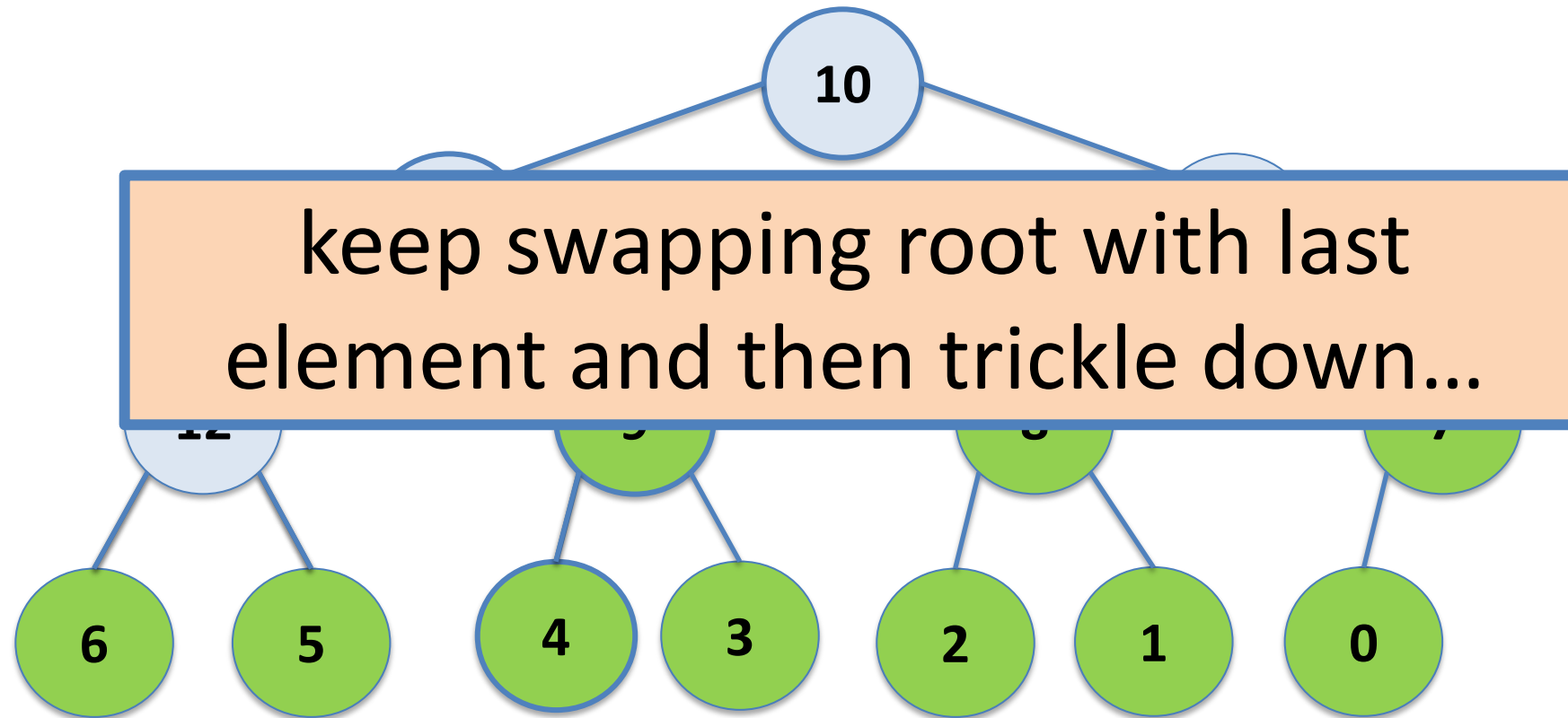
Sorting a Heap

10	11	13	12	9	8	7	6	5	4	3	2	1	0	
----	----	----	----	---	---	---	---	---	---	---	---	---	---	--



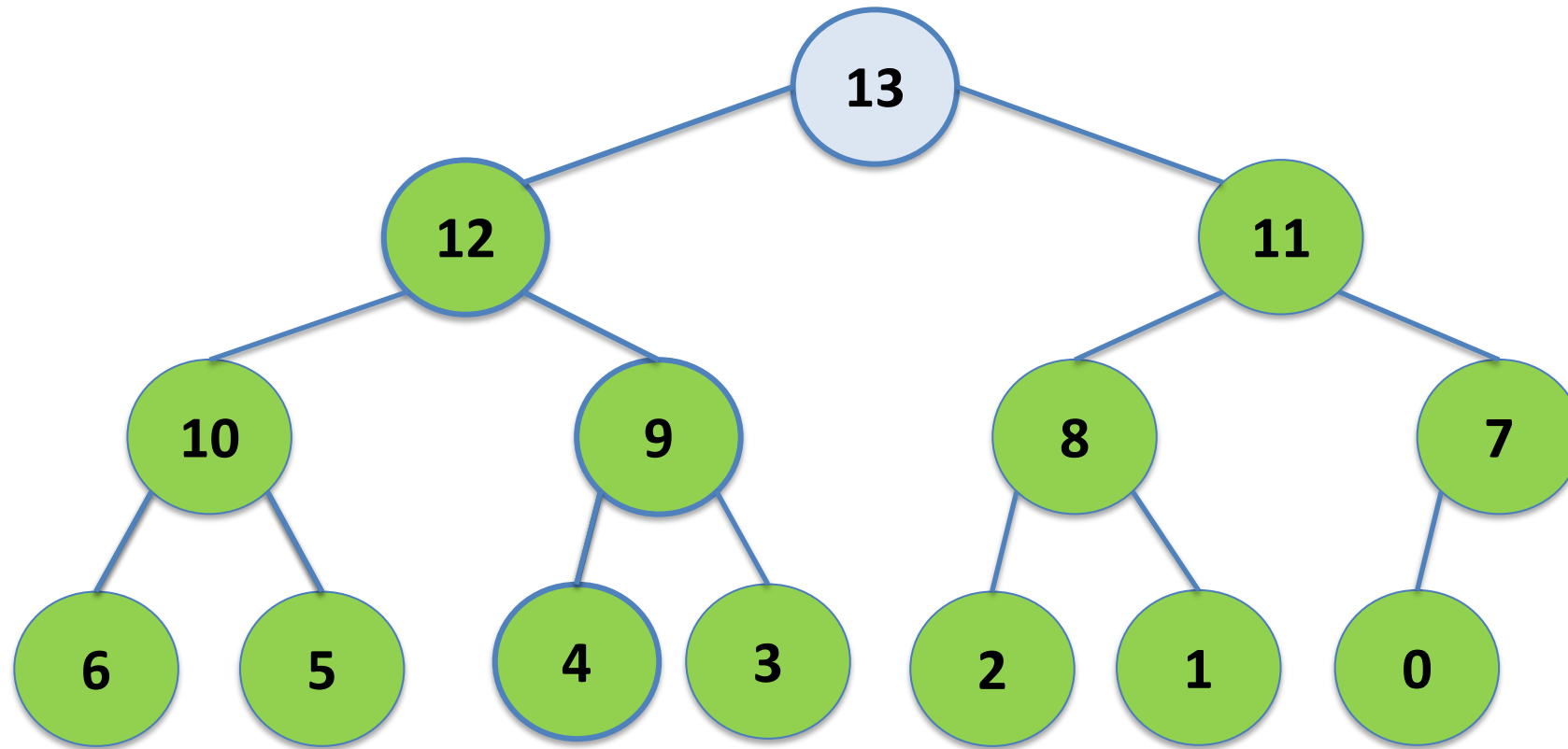
Sorting a Heap

10	11	13	12	9	8	7	6	5	4	3	2	1	0	
----	----	----	----	---	---	---	---	---	---	---	---	---	---	--

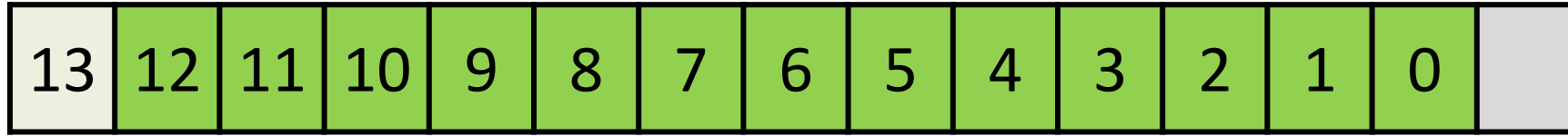


Sorting a Heap

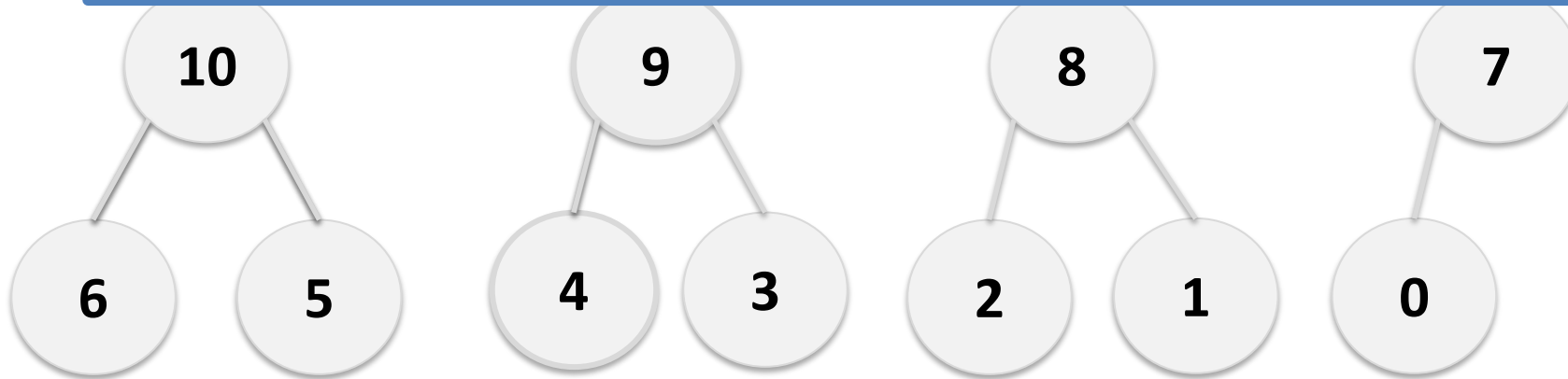
13	12	11	10	9	8	7	6	5	4	3	2	1	0	
----	----	----	----	---	---	---	---	---	---	---	---	---	---	--



Sorting a Heap



The array is sorted in
reverse order



Heapsort

Heapsort(A):

 heapify(A)

 for i from n-1 down to 2

 swap A[i] and A[0]

 trickleDown(0)

reverse(A)

Heapsort

Heapsort(A):

 heapify(A)

 for i from n-1 down to 2

 swap A[i] and A[0]

 trickleDown(0)

reverse(A)

$O(n)$

Heapsort

Heapsort(A):
 heapify(A)

for i from n-1 down to 2
 swap A[i] and A[0]
 trickleDown(0)

reverse(A)

$O(n \log n)$

$O(n)$

Heapsort

Heapsort(A):

heapify(A)

Array-based
binary heap

$O(n)$

for i from n-1 down to 2

 swap A[i] and A[0]

 trickleDown(0)

$O(n \log n)$

reverse(A)

$O(n)$

Heapsort

Theorem 11.4: Heapsort runs in $O(n \log(n))$ time and performs at most $2n \log(n) + O(n)$ comparisons.

Heapsort runs in-place.

Heapsort is *not* stable.